EXAMPLE OF A SIMPLE ALGORITHM

To introduce the Pearson Edexcel pseudocode, here is a simple written algorithm that asks the user to input two numbers and then outputs the result of adding them together.

WRITTEN DESCRIPTION

ALGORITHM FOR ADDING TWO NUMBERS

Enter first number.
Enter second number.
Calculate total by adding first and second numbers.
Output total.

FLOWCHART



▲ **Figure 1.4** Flowchart showing the adding of two numbers

SUBJECT VOCABULARY

variable a 'container' used to store data. The data stored in a variable is referred to as a value. The value stored in a variable is not fixed. The same variable can store different values during the course of a program and each time a program is run

identifier a unique name given to a variable or a constant. Using **descriptive** names for variables makes code much easier to read

arithmetic operator an **operator** that performs a calculation on two numbers

GENERAL VOCABULARY

operator a character that represents an action, e.g. 'x' represents a multiplication and '/' a division

descriptive describing something clearly

PSEUDOCODE

ALGORITHM FOR ADDING TWO NUMBERS

```
SEND 'Please enter the first number' TO DISPLAY
RECEIVE firstNumber FROM KEYBOARD
SEND 'Please enter the second number' TO DISPLAY
RECEIVE secondNumber FROM KEYBOARD
SET total TO firstNumber + secondNumber
SEND total TO DISPLAY
```

The pseudocode gives clear step-by-step instructions that the computer will be expected to carry out. It also introduces some important programming concepts.

- The numbers entered by the user are stored in two **variables** with the **identifiers** `firstNumber` and `secondNumber`.
- The result of adding the numbers together is stored in the variable `total`.
- Text has to be placed in quotation marks (single or double) if it is to be displayed. For example, 'Please enter the first number' (or "Please enter the first number").
- Quotation marks are not used if a variable is to be displayed. If they were, the word 'total' in the last instruction would be displayed instead of the number it represents.
- **Arithmetic operators** are used to perform calculations. Table 1.1 shows the arithmetic operators.

ARITHMETIC OPERATORS

OPERATOR	FUNCTION	EXAMPLE
+	Addition: add the values together.	8 + 5 = 13 myScore1 + myScore2
–	Subtraction: subtract the second value from the first.	17 – 4 = 13 myScore1 – myScore2
*	Multiplication: multiply the values together.	6 * 9 = 54 numberBought * price
/	Real division: divide the first value by the second value and return the result including decimal places.	13 / 4 = 3.25 totalMarks/numberTests
DIV	**Quotient**: like division, but it only returns the whole number or *integer*.	13 DIV 4 = 3 totalMarks DIV numberTests
MOD	**Modulus**/modulo: this will return the remainder of a division.	13 / 4 = 3 remainder 1 Therefore 13 MOD4 = 1
^	Exponentiation: this is for 'to the **power** of'.	3 ^ 3 = 27 It is the same as writing 3^3

▲ **Table 1.1** Arithmetic operators

VARIABLES AND CONSTANTS

KEY POINT

When you chose a variable name, it should be descriptive of the data it will hold, e.g. distance or length. Long variable names are easier to misspell!

SUBJECT VOCABULARY

constant a 'container' that holds a value that never changes; like variables, constants have unique identifiers

GENERAL VOCABULARY

quotient a result found by dividing one quantity by another

modulus the remainder after the division of one number by another

power the small number written to the right and above another number to show how many times it should be multiplied by itself

Variables play an important role in algorithms and programming. The value stored by a variable can change as a program is running. Variables are extremely useful in programming because they make it possible for the same program to process different sets of data.

A **constant** is the opposite of a variable. It is a 'container' that holds a value that always stays the same. Constants are useful for storing fixed information, such as the value of pi, the number of litres in a gallon or the number of months in a year.

Each variable and constant in an algorithm has to have a unique identifier. It is important to choose descriptive names for identifiers. This will make your code much easier to read. For example, a variable to hold a user's first name could be given the identifier `firstName` to show the data it contains. If it were given the identifier `X` instead, it wouldn't clearly show what data it contained.

NAMING CONVENTIONS FOR VARIABLES AND CONSTANTS

It is sensible to write identifiers in the same way throughout an algorithm. A common method is to use 'camel case' for compound words (e.g. `firstName`, `secondName`) with no space between words and the second word starting with a capital letter. Alternatively, you could capitalise the first letter of both words, e.g. `FirstName, SecondName`, or separate the words with an underscore, e.g. `first_name, second_name`, known as 'snake case'.

SUBJECT VOCABULARY

initialise to set variables to their starting values at the beginning of a program or subprogram

initialisation the process of assigning an initial value to a variable

assignment statement the SET…TO command is used to initialise variables in pseudocode, for example:

```
SET anotherGo TO 0
SET correct TO False
```

You can put an initial value into a variable by:

- initialising it when the program is run (e.g. `SET total TO 0`, in pseudocode)
- reading a value from a keyboard or other device (e.g. `RECEIVE admissionCharge FROM (INTEGER) KEYBOARD`).

Once a variable has been **initialised** an **assignment statement** is used to change its value (e.g. `SET total TO total + admissionCharge`).

In Python this would be:

```
total = 0
total = total + admissionCharge
```

In Java this would be:

```
Scanner scan = new Scanner(System.in);
int admissionCharge = scan.nextInt();
int total = 0;
total = total + admissionCharge;
```

In C#, variables must be declared before use. When you declare a variable, you need to state the data type that the variable will store, for example:

```
// declare a variable called total that will be used to store floating point numbers and assign the value 0.0 to it
float total = 0.0;
// add the value stored in the variable admissionsCharge to the total
total = total + admissionsCharge;
```

GENERAL VOCABULARY

intended designed for a certain purpose

If a variable, such as a loop counter, is **intended** to hold a running total, then it should always be initialised to a starting value. Some programming languages won't execute if the programmer fails to do this; others will do so but may well produce some unexpected results.

SKILLS CRITICAL THINKING, DECISION MAKING

ACTIVITY 1

DATA TYPES

1 Investigate what data types are available in the high-level language you are studying. Produce a table similar to Table 2.1 to give a summary of your findings.

2 What do you think is an appropriate data type for each of these items?
   a the test score of an individual learner
   b the average score for a group of learners
   c whether or not the pass mark for the test has been achieved.

3 Look back over the algorithms you wrote in Unit 1 and find instances of variable initialisation.

SUBJECT VOCABULARY

type coercion the process of converting the value stored in a variable from one data type to another

TYPE COERCION

Sometimes the data type of a variable gets changed during program execution. This is known as **type coercion**. For example, if an integer value and a real value are used in an arithmetic operation, the result will always be a real.

In Python, type coercion is done automatically, for example, the output from the following program is 3.25.

```
x = 1
y = 2.25
z = x + y
print(z)
```

Type coercion is also automatic in Java:

```
int x = 1;
double y = 2.25;
double z = x + y;
System.out.print(z);
```

In C# this is generally referred to as casting. As with Python this can be automatic for many data type conversions and is known as implicit casting. For more complicated conversions you may need to research explicit casting.

One frequently used type of explicit casting is converting from a string to a numeric value. This is used when getting input from a user as `Console.Readline`. For example, the following C# code will output 3.25 and demonstrates automatic type coercion.

```
int x;                  // declares a variable called x that will store integers
double y, z;            // declares two variables called y and z that will store
                        floating point numbers
x = 1;
y = 2.25;
z = x + y;
Console.WriteLine(z);
```

SKILLS CRITICAL THINKING, PROBLEM SOLVING, DECISION MAKING

ACTIVITY 2

MONITORING VISITOR NUMBERS



A theme park uses a program to monitor the number of people entering and exiting the park. The maximum number of visitors at any one time must not exceed 10 000. When the number of people in the park reaches the maximum, a 'Park Full' message is displayed at the entrance gate. Children can visit the park free of charge. Adults must pay AED 125 admission. The program records the amount of money collected at the gate.

1 What are the variables needed in the program?

2 Select an appropriate data type for each variable and constant.

COMMAND SEQUENCE, SELECTION AND ITERATION

In Unit 1 you learnt that the three key building blocks of algorithms are command sequence, selection and iteration. In this unit, you will have the opportunity to implement these constructs in the high-level programming language you are studying.

SKILLS CRITICAL THINKING, PROBLEM SOLVING, CREATIVITY

ACTIVITY 3

UNDERSTANDING ALGORITHMS

Read the following algorithm written in pseudocode and then answer the questions below.

```
RECEIVE number1 FROM (INTEGER) KEYBOARD
RECEIVE number2 FROM (INTEGER) KEYBOARD
SET result1 TO number1 / number2
SEND result1 TO DISPLAY
SET result2 TO number1 MOD number2
SEND result2 TO DISPLAY
SET result3 TO number1 DIV number2
SEND result3 TO DISPLAY
```

1 What does this algorithm do?

2 What is the output of the algorithm, given the following inputs:
   a 4, 2
   b 10, 3
   c 20, 6?

Implement this algorithm in the high-level language you are studying.

SELECTION

The selection construct is used to create a branch in a program. The computer selects which branch to follow based on the outcome of a condition, using an `IF...THEN...ELSE` statement. For example, in pseudocode:

```
IF day = 'Saturday' OR day = 'Sunday' THEN
  SET alarm TO 11
ELSE
  SET alarm TO 8
END IF
```

SUBJECT VOCABULARY

nested IF statement a nested IF statement consists of one or more IF statements placed inside each other. A nested IF is used where there are more than two possible courses of action

A standard `IF...THEN...ELSE` statement provides two alternatives. If there are more than two, then in Pearson Edexcel pseudocode a nested `IF` must be used. However, many high-level programming languages have an additional built-in selection construct that does away with the need for a **nested IF statement**.



▲ Calculators have a number of useful functions

SKILLS REASONING

SKILLS CRITICAL THINKING

SKILLS PROBLEM SOLVING

2 Develop an algorithm for a simple calculator and code it in a high-level language that:
   - **a** allows the user to choose from these options: addition, subtraction, division and multiplication
   - **b** prompts the user to input two numbers
   - **c** performs the calculation and displays the result
   - **d** offers the user the option of performing another calculation.

CHECKPOINT

Strengthen

S1 Why it is important to make your code easy to read?

S2 Outline the four techniques that a programmer should use to make code easy to read.

Challenge

C1 Revisit the programs you have already written. Do you still understand what they do and how they work? If not, try to improve their readability.

KEY POINT

Readability of code is important. You should always make your code easy to read.

How confident do you feel about your answers to these questions? If you're not sure you answered them well, look again at Table 2.5.

SUMMARY

- Using comments, descriptive names, indentation and white space makes code easier to read.
- Producing readable code makes it easier to understand what a program does and how it does it.